



## Chapter 2: Operating-System Structures

# Chapter 2: Operating-System Structures

- Operating System Services
- User Operating System Interface
- System Calls
- Types of System Calls
- System Programs
- **Operating System Design and Implementation**
- Operating System Structure

# Operating System Design and Implementation

- Design and Implementation of OS not “**solvable**”, but some approaches have proven successful
- Internal structure of different Operating Systems can vary widely
- Start the design by **defining goals** and specifications
- Affected by choice of hardware, type of system

# Operating System Design and Implementation

- **User** goals and **System** goals
  - **User goals** – operating system should be easy to use, easy to learn, reliable, safe, and fast
  - **System goals** – operating system should be easy to design, implement, and maintain, as well as flexible, reliable, error-free, and efficient
- Specifying and designing an OS is highly creative task of software engineering.

# Implementation

- **Much variation**
  - Early OSES in assembly language
  - Then system programming languages like Algol, PL/1
  - Now C, C++
- **Actually usually a mix of languages**
  - Lowest levels in assembly “Compatible for H/W”
  - Main body in C
  - Systems programs in C, C++, scripting languages like PERL, Python, shell scripts

# Chapter 2: Operating-System Structures

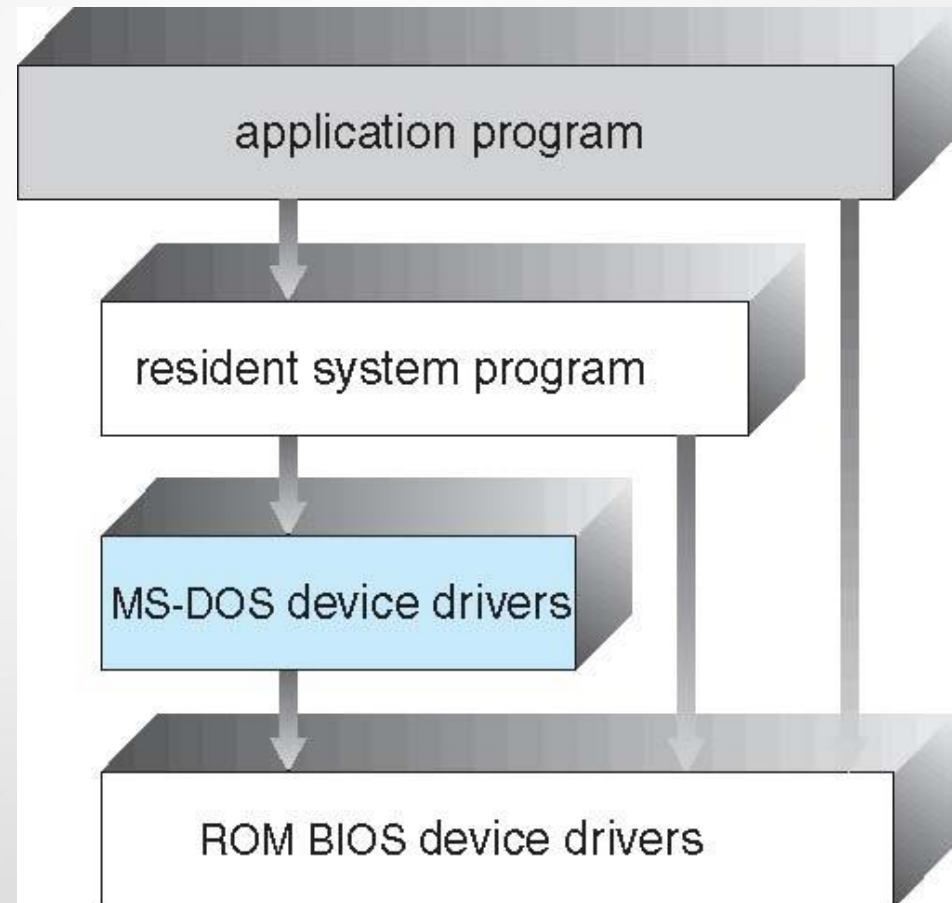
- Operating System Services
- User Operating System Interface
- System Calls
- Types of System Calls
- System Programs
- Operating System Design and Implementation
- **Operating System Structure**

# Operating System Structure

- General-purpose OS is very large program
- Various ways to structure ones
  - Simple structure – MS-DOS
  - More complex -- UNIX
  - Layered
  - Microkernel -Mach

# Simple Structure -- MS-DOS

- MS-DOS – written to provide the most functionality in the least space
  - MS-DOS has some structure, its interfaces and levels of functionality are not well separated





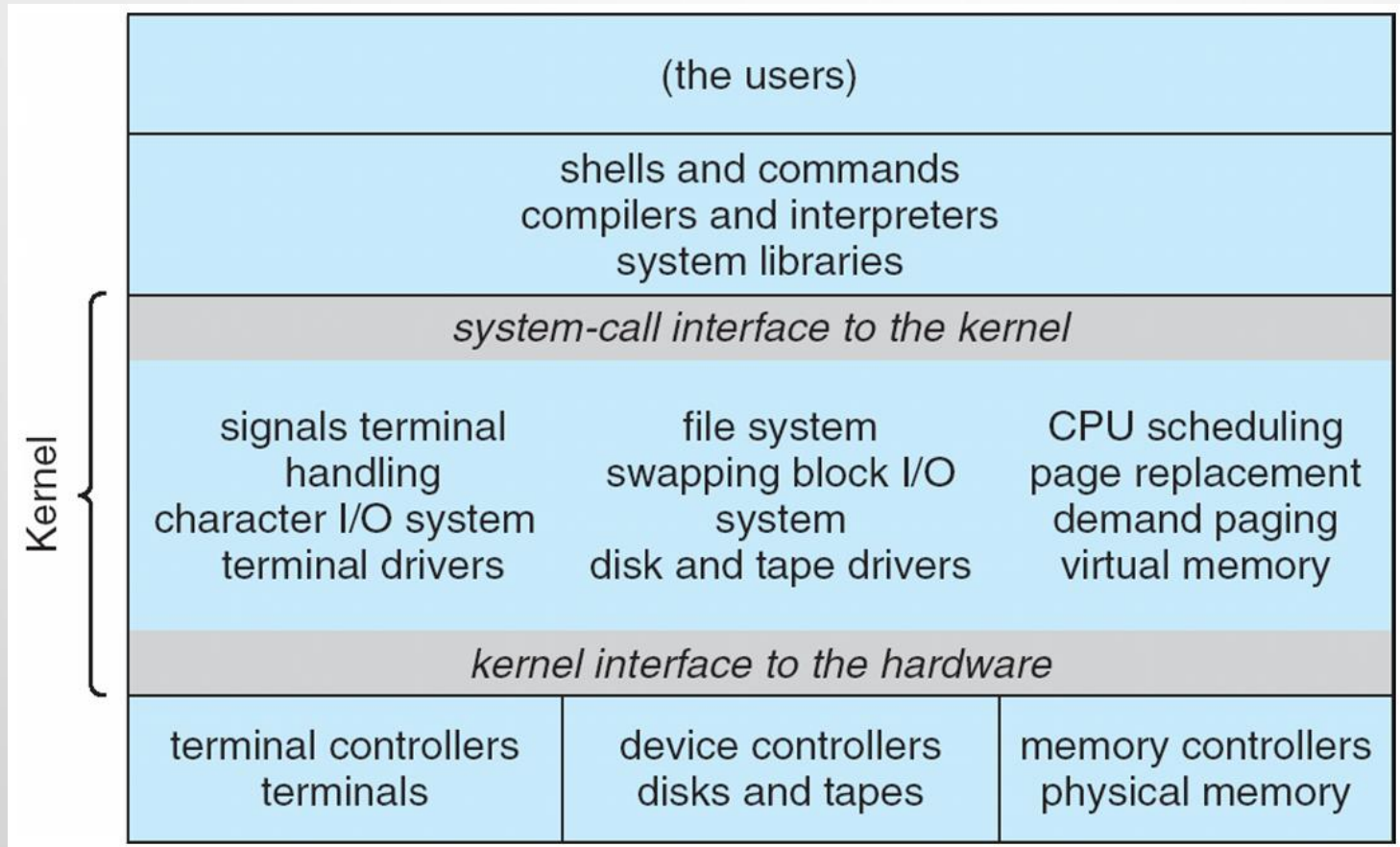
# Non Simple Structure -- UNIX

UNIX – limited by hardware functionality, the original UNIX operating system had limited structuring.

The UNIX OS consists of two separable parts

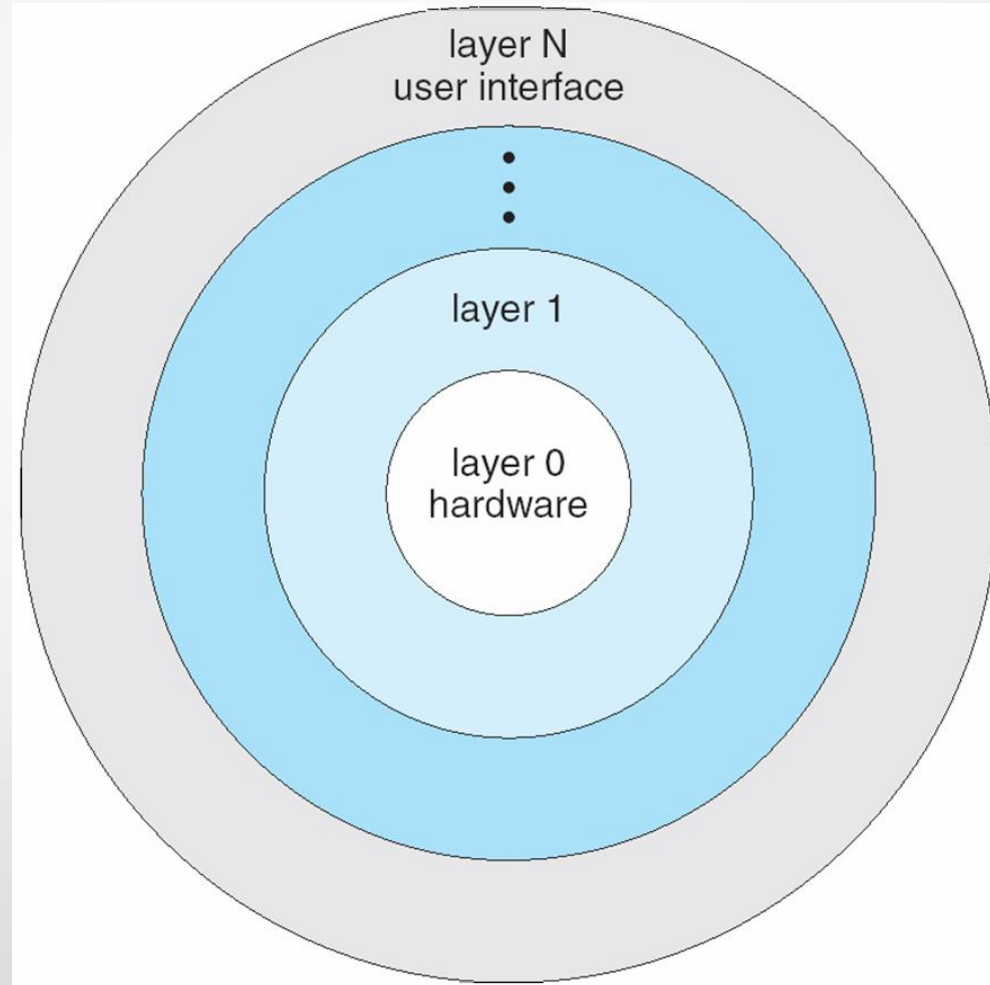
- Systems programs
- The kernel
  - Consists of everything below the system-call interface and above the physical hardware
  - Provides the file system, CPU scheduling, memory management, and other operating-system functions; a large number of functions for one level

# Traditional UNIX System Structure



# Layered Approach

- The operating system is divided into a number of layers (levels), each built on top of lower layers. The bottom layer (layer 0), is the hardware; the highest (layer N) is the user interface.
- With modularity, layers are selected such that each uses functions (operations) and services of only lower-level layers

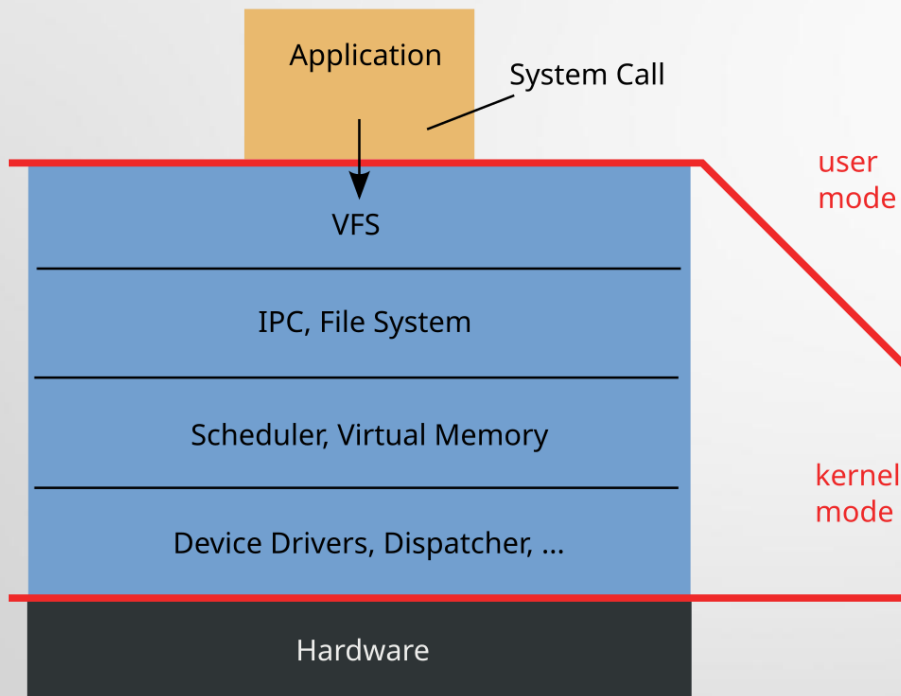


# Microkernel System Structure

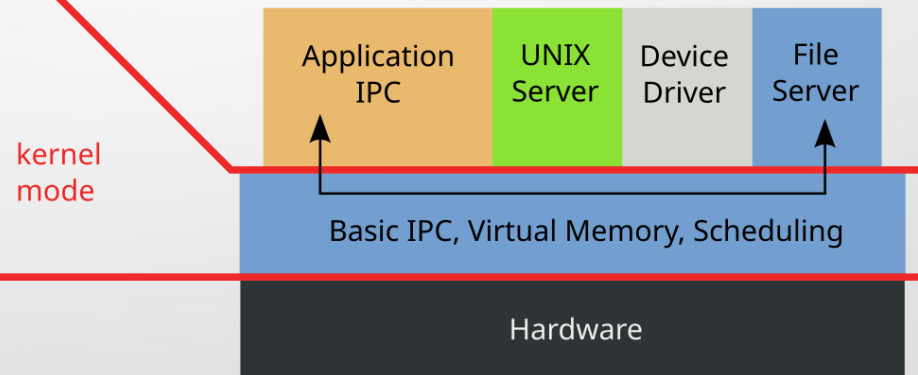
- Moves as much from the kernel into user space
- **Mach** example of **microkernel**
  - Mac OS X kernel (**Darwin**) partly based on Mach

# Microkernel System Structure

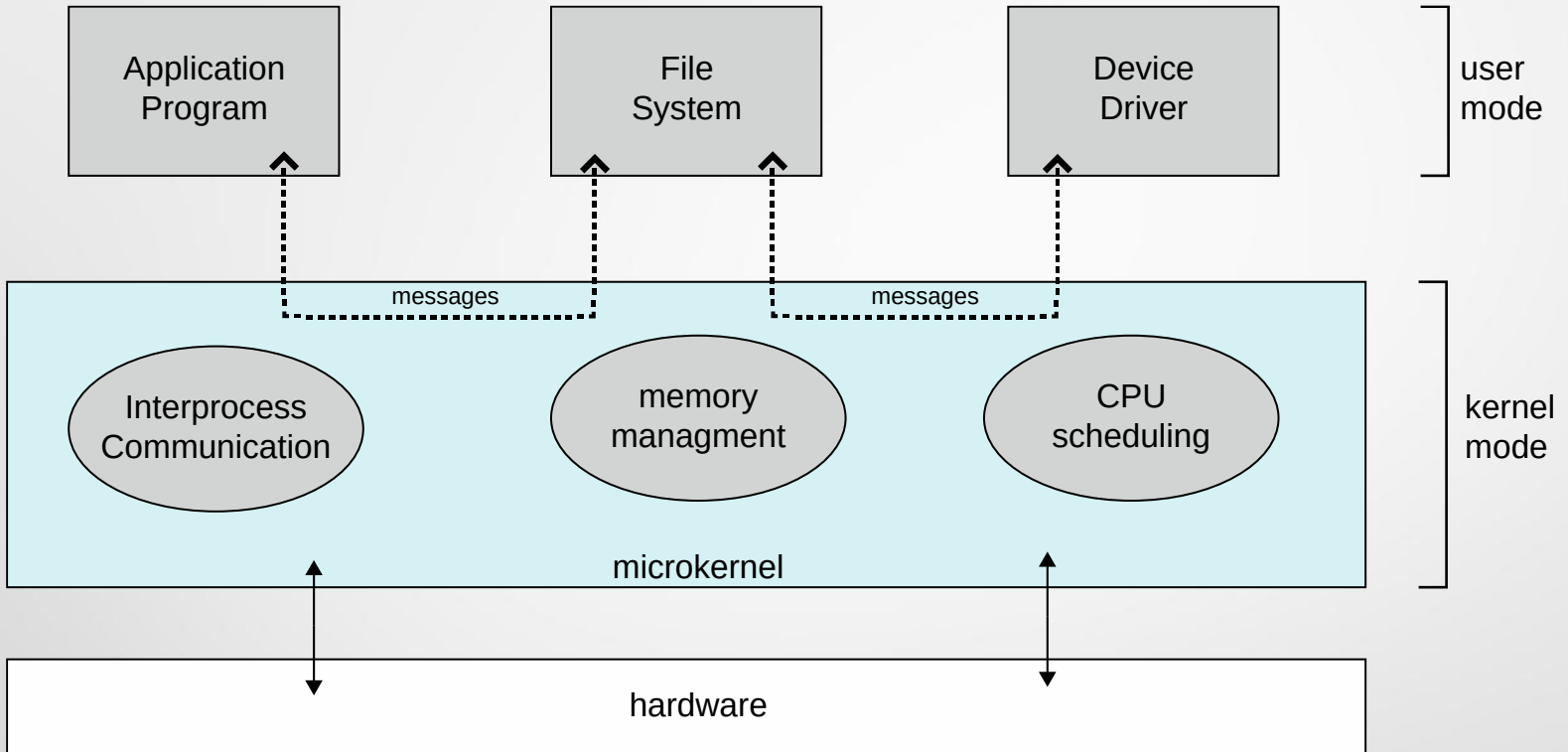
Monolithic Kernel  
based Operating System



Microkernel  
based Operating System



# Microkernel System Structure



End of Chapter 2